

Disciplina: DCA 3703 - PROGRAMAÇÃO PARALELA

Professor: Samuel Xavier De Souza

Nome: José Felix Rodrigues Anselmo

Tarefa 3

1 - Enunciado

Implemente três laços em C para investigar os efeitos do paralelismo ao nível de instrução (ILP): 1) inicialize um vetor com um cálculo simples; 2) some seus elementos de forma acumulativa, criando dependência entre as iterações; e 3) quebre essa dependência utilizando múltiplas variáveis. Compare o tempo de execução das versões compiladas com diferentes níveis de otimização (O0, O2, O3) e analise como o estilo do código e as dependências influenciam o desempenho.

2 - Metodologia

Para esse projeto foi implementado o código em C, o primeiro passo foi criar um laço seguindo a orientação da tarefa aplicando uma operação simples $v[i] = (\text{double})i * 0.5 + 1.0$, para cada laço foi usado duas variáveis de tempo `tinumerolaço` e `tfnumerolaço` para representar o tempo de execução de cada laço e ao final é feito a subtração das somas realizadas durante o processo como requisitado na tarefa. Os códigos foram executados da seguinte forma:

```
gcc -O0 main.c -o main1
```

```
gcc -O2 main.c -o main2
```

```
gcc -O3 main.c -o main3
```

2.2 Algoritmo

Etapas do algoritmo:

1. Definir o tamanho do vetor e o número de repetições.
2. Alocar o vetor v.
3. Preencher o vetor com valores calculados a partir do índice.
4. Executar o Laço 1, calculando cada posição do vetor e medir seu tempo.
5. Executar o Laço 2, realizando a soma sequencial dos elementos e medir seu tempo.
6. Executar o Laço 3, dividindo a soma em quatro acumuladores independentes e medir seu tempo.
7. Comparar os resultados das duas formas de soma.
8. Exibir os tempos de execução e os resultados.
9. Liberar a memória do vetor.

2.3 - Ambiente de Execução

Componente	Especificação
Notebook	Lenovo IdeaPad 3 15ALC6
SO	Ubuntu 26.04 LTS
Kernel	Linux 7.0.0-29-generic
CPU	AMD Ryzen 5 5500U — 6C/12T
Arquitetura	x86_64
RAM	9,05 GiB
Desktop	GNOME 50.1
Shell	Zsh 5.9
Terminal	Kitty 0.45.0
Compilador	GCC

3 - Código Fonte

```
#include <stdio.h>
#include <stdlib.h>
```

```
#include <time.h>

#define TAM 60000000UL
#define REPS 5

static double medirTempo(void) {
    struct timespec ts;

    clock_gettime(CLOCK_MONOTONIC, &ts);

    return ts.tv_sec + ts.tv_nsec * 1e-9;
}

int main(void) {
    double *v = malloc(TAM * sizeof(double));

    // Laço 1:
    double ti1 = medirTempo();
    for (unsigned long r = 0; r < REPS; r++) {
        for (unsigned long i = 0; i < TAM; i++) {
            v[i] = (double)i * 0.5 + 1.0;
        }
    }

    double tf1 = medirTempo();
    printf("Laço 1 %.4f s\n", (tf1 - ti1) / REPS);

    // Laço 2: soma acumulativa
    double somaAcumulativa = 0.0;
    double ti2 = medirTempo();
    for (unsigned long r = 0; r < REPS; r++) {
        somaAcumulativa = 0.0;
```

```

        for (unsigned long i = 0; i < TAM; i++) {
            somaAcumulativa += v[i];
        }
    }

    double tf2 = medirTempo();

    printf("Laco 2 somaAcumulativa %.4f s (soma = %.3f)\n", (tf2 - ti2) / REPS,
somaAcumulativa);

//Laço 3

    double somaQuebrada = 0.0;

    double ti3 = medirTempo();

    for (unsigned long r = 0; r < REPS; r++) {

        double s0 = 0.0, s1 = 0.0, s2 = 0.0, s3 = 0.0;

        unsigned long i;

        for (i = 0; i + 4 <= TAM; i += 4) {

            s0 += v[i];

            s1 += v[i + 1];

            s2 += v[i + 2];

            s3 += v[i + 3];

        }

        for (; i < TAM; i++) {

            s0 += v[i];

        }

        somaQuebrada = (s0 + s1) + (s2 + s3);

    }

    double tf3 = medirTempo();

    printf("Laco 3 SomaQuberada %.4f s (soma = %.3f)\n", (tf3 - ti3) / REPS,
somaQuebrada);

```

```

printf("\n Diferenca somaAcumulativa - somaQuebrada = %g (esperado ~0, so erro de
arredondamento)\n", somaAcumulativa - somaQuebrada);

free(v);

return 0;
}

```

4 - Resultado

Ao executar os testes um após o outro foi obtido a seguinte saída que pode ser visualizada na figura 1:

```

> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_3
> ./main1
Laco 1 0.3124 s
Laco 2 (soma serial).....: 0.2752 s (soma = 900000045000000.000)
Laco 3 (soma c/ 4 acumuladores): 0.0818 s (soma = 900000045000000.000)

Diferenca somaAcumulativa - somaQuebrada = 0 (esperado ~0, so erro de arredondamento)
> ./main2
Laco 1 0.1619 s
Laco 2 (soma serial).....: 0.0805 s (soma = 900000045000000.000)
Laco 3 (soma c/ 4 acumuladores): 0.0472 s (soma = 900000045000000.000)

Diferenca somaAcumulativa - somaQuebrada = 0 (esperado ~0, so erro de arredondamento)
> ./main3
Laco 1 0.0780 s
Laco 2 (soma serial).....: 0.0803 s (soma = 900000045000000.000)
Laco 3 (soma c/ 4 acumuladores): 0.0484 s (soma = 900000045000000.000)

Diferenca somaAcumulativa - somaQuebrada = 0 (esperado ~0, so erro de arredondamento)

```

Figura 1: Execução no terminal

Com isso foi montada a seguinte tabela 1 :

Otimização	Laço 1(iniciar)	Laço 2 (SomaAcumula da)	Laço 3 (SomaQuebrada)
-O0	0.3124 s	0.2752 s	0.0818 s
-O2	0.1619 s	0.0805 s	0.0472 s
-O3	0.0780 s	0.0803 s	0.0484 s

Tabela 1: Tempos de execução

5 - Conclusão

Como podemos vê os testes realizados nos mostram várias coisas interessante no que diz respeito a otimização da compilação, no pior caso com -O0 podemos vê que nos 3 laços apresentam os piores tempos, mesmo para o laço 3 no qual é dívida a soma em 4 outras somas, a divisão da soma em quatro acumuladores independentes reduz a dependência entre as iterações e permite ao processador explorar melhor o paralelismo ao nível de instrução.

A Partir da -O2 podemos ver uma redução significativa do tempo e isso diz respeito às otimizações realizadas a nível de compilação que mesmo no caso onde a dependência de dados para acumulação o processo consegue se sair melhor.

Outro ponto interessante é que mesmo usando uma otimização mais agressiva -O3 não temos grandes melhoras para a SomaAcumulada isso mostra que existem limitações causadas pela dependência entre as operações, fazendo com que nem todas as otimizações do compilador consigam gerar ganhos significativos nesse caso.

Esse experimento é muito interessante, pois apresenta na prática vários conceitos vistos em Arquitetura de Computadores e Programação Paralela, mostrando que a forma como o código é organizado pode influenciar diretamente seu desempenho. Ao reduzir as dependências entre as operações e utilizar múltiplos acumuladores, é possível aumentar o paralelismo ao nível de instrução e obter uma melhor execução do programa.